

Documentație proiect

Numele lucrării: Photo Booth

Realizator: Inginer Uleanu Dragoș - Florin

1. Introducere

Aplicația Photo Booth a fost dezvoltată cu scopul de a oferi utilizatorilor o modalitate simplă și securizată de gestionare a imaginilor. Sistemul reprezintă o arhitectură clasică (server-client). În mod concret sistemul este alcătuit din trei componente:

- i) O aplicație client dezvoltată în Unity folosind limbajul de programare C#
- ii) Un server dezvoltat folosind ASP.NET
- iii) O bază de date Microsoft SQL Server

Aplicația permite înregistrarea și autentificarea utilizatorilor, încărcarea imaginilor, organizarea acestora pe baza unui subiect definit de utilizator, duplicarea imaginilor și aplicarea unor filtre grafice predefinite.

Pentru protejarea datelor, imaginile sunt stocate pe server în formă criptată și pot fi decriptate numai prin utilizarea parolei de criptare introduse de utilizator.

Comunicarea dintre client și server este realizată prin intermediul unui API REST, iar accesul la operațiile asupra imaginilor este protejat prin autentificarea bazată pe JSON Web Token.

2. Scopul aplicației

Scopul proiectului este dezvoltarea unei aplicații client-server pentru gestionarea securizată a imaginilor.

Aplicația urmărește separarea responsabilităților între interfața utilizatorului, logica serverului și persistarea datelor.

Clientul Unity este responsabil pentru interacțiunea cu utilizatorul, serverul ASP.NET CORE gestionează logica aplicației și securitatea, iar baza de date SQL Server păstrează informațiile despre utilizatori și metadatele imaginilor.

Un obiectiv important al aplicației este protejarea imaginilor stocate. Înainte de salvarea pe server, conținutul imaginilor este criptat folosind o cheie derivată din parola introdusă de utilizator. Astfel fișierele salvate pe disc nu pot fi vizualizate direct fără procesul de decriptare realizat de aplicație.

3. Cerințe Funcționale

În cadrul proiectului au fost definite o serie de cerințe funcționale referitoare la aplicația client, componenta server, persistența datelor și versionarea codului. Aplicația dezvoltată urmărește îndeplinirea integrală a acestor cerințe.

3.1 Aplicația Client

Aplicația client trebuie să ofere utilizatorului posibilitatea de a gestiona o colecție de imagini. Principalele funcționalități solicitate sunt:

Încărcarea imaginilor – utilizatorul trebuie să poată selecta o imagine disponibilă de pe sistemul local și să o încarce în aplicație

Duplicarea imaginilor – O imagine existentă trebuie să poată fi duplicată, rezultând o nouă imagine disponibilă în galerie

Aplicarea filtrelor – Utilizatorul trebuie să poată selecta o imagine și să aleagă un filtru dintr-o listă prestabilită. Aplicare filtrului nu trebuie să înlocuiască imaginea originală, ci să genereze o nouă imagine care va fi disponibilă în galerie.

Gruparea imaginilor pe subiecte – Utilizatorul trebuie să poată defini un subiect pentru o imagine, precum „Revelion 2025” și să poată vizualiza imaginile aparținând unui anumit subiect.

Stocarea criptată a imaginilor – Imaginile trebuie salvate în formă criptată utilizând o cheie bazată pe o parolă introdusă de utilizator. Pentru vizualizare, aplicația trebuie să permită decriptarea imaginii folosind parola corespunzătoare.

3.2 Componenta Server

Soluția trebuie să conțină o componentă server care comunică cu aplicația client. Serverul trebuie să deservească cel puțin două dintre funcționalitățile definite pentru client.

Pentru realizarea comunicației client-server este necesară dezvoltarea unui **API** care să poată fi consumat de aplicația client.

Componenta server trebuie, de asemenea, să permită **înregistrarea și autentificarea utilizatorilor**, astfel încât accesul la funcționalitățile aplicației să poată fi asociat unui utilizator.

3.3 Persistența datelor

Soluția trebuie să utilizeze o **bază de date** pentru stocarea persistentă a informațiilor necesare funcționării aplicației.

Asupra bazei de date trebuie să poată fi efectuate operații de tip **CRUD**.

3.4 Versionarea codului

Pe parcursul dezvoltării trebuie utilizat un sistem de versionare a codului, care să permită urmărirea modificărilor realizate asupra proiectului și gestionarea diferitelor versiuni ale acestuia.

4. Arhitectura aplicației

Aplicația **Photo Booth** a fost dezvoltată utilizând o arhitectură de tip client-server prin care interfața utilizatorului este separată de logica aplicației și de mecanismele de persistență a datelor. Soluția este alcătuită din trei componente principale: aplicația client dezvoltată în Unity, componenta server realizată cu ASP.NET Core Web API și baza de date Microsoft SQL Server.

Pe lângă acestea, serverul gestionează și stocarea fizică a imaginilor, acestea fiind salvate pe disc numai după criptare.

4.1 Arhitectura client-server

Arhitectura aleasă separă responsabilitățile aplicației între client și server. Clientul este responsabil în principal pentru interacțiunea cu utilizatorul și prezentarea informațiilor, în timp ce serverul implementează logica principală a aplicației.

Clientul nu comunică direct cu baza de date și nu accesează direct fișierele stocate pe server. Toate operațiile sunt realizate prin intermediul API-ului.

Această abordare permite ca operațiile importante, precum autentificarea, autorizarea, criptarea și accesul la baza de date, să fie realizate la nivelul serverului și nu direct de către client.

4.2 Clientul Unity

Componenta client a fost dezvoltată utilizând **Unity** și reprezintă punctul principal de interacțiune dintre utilizator și sistem.

Clientul oferă interfețele necesare pentru înregistrarea și autentificarea utilizatorului, precum și o galerie prin intermediul căreia acesta poate gestiona imaginile asociate contului său.

Printre operațiile disponibile la nivelul clientului se numără selectarea și încărcarea unei imagini, vizualizarea imaginilor, duplicarea acestora, aplicarea filtrelor, ștergerea și filtrarea galeriei în funcție de subiect.

Comunicarea cu serverul este realizată prin request-uri **HTTP/HTTPS**. Pentru aceasta, clientul utilizează **UnityWebRequest**. În funcție de operație, informațiile sunt transmise sub formă de **JSON** sau **multipart/form-data**.

Clientul nu implementează acces direct la SQL Server. Orice operație asupra datelor trebuie să treacă prin API.

4.3 Componenta ASP.NET Core Web API

Serverul reprezintă componenta centrală a sistemului și a fost dezvoltat utilizând ASP.NET Core Web API.

Aceasta expune o serie de endpoint-uri REST care pot fi accesate de aplicația Unity. Serverul este responsabil pentru procesarea request-urilor primite de la client și pentru realizarea operațiilor solicitate.

Printre responsabilitățile serverului se află:

- Înregistrarea și autentificarea utilizatorilor,
- Generarea și verificarea token-urilor JWT

- Autorizarea accesului la imagini

Prin această structură, logica aplicației este centralizată la nivelul serverului, iar clientul este separat de implementarea internă a bazei de date și a mecanismelor de securitate.

4.4 Baza de date SQL Server

Pentru persistența datelor este utilizat **Microsoft SQL Server**, iar comunicarea dintre API și baza de date este realizată cu ajutorul Entity Framework Core.

Baza de date conține informațiile referitoare la utilizatorii aplicației și metadatele necesare gestionării imaginilor.

Pentru fiecare imagine sunt păstrate informații precum identificatorul acesteia, subiectul, utilizatorul căruia îi aparține, calea către fișier, tipul de conținut și parametrii criptografici necesari procesului de decriptare.

Conținutul efectiv al imaginilor nu este blocat în baza de date. În schimb, baza de date păstrează informațiile necesare pentru identificarea fișierului criptat corespunzător.

4.5 Stocarea fișierelor criptate

Una dintre particularitățile arhitecturii aplicației este separarea dintre metadatele unei imagini și conținutul propriu-zis al acesteia.

Atunci când utilizatorul încarcă o imagine, aceasta nu este salvată direct pe server în forma originală. Serverul citește conținutul fișierului și îl criptează înainte de scrierea pe disc.

Fișierul rezultat este salvat în directorul destinat imaginilor încărcate, în timp ce baza de date păstrează calea către acesta și informațiile criptografice necesare decriptării.

Astfel, simpla ccesare a fișierului de pe server nu permite vizualizarea imaginii originale.

4.6 Fluxul general al datelor

Fluxul datelor diferă în funcție de operația efectuată. În cazul unei încărcări, de exemplu, utilizatorul selectează imaginea în clientul Unity, definește subiectul și introduce parola de criptare. Clientul transmite datele către API, împreună cu token-ul de autentificare.

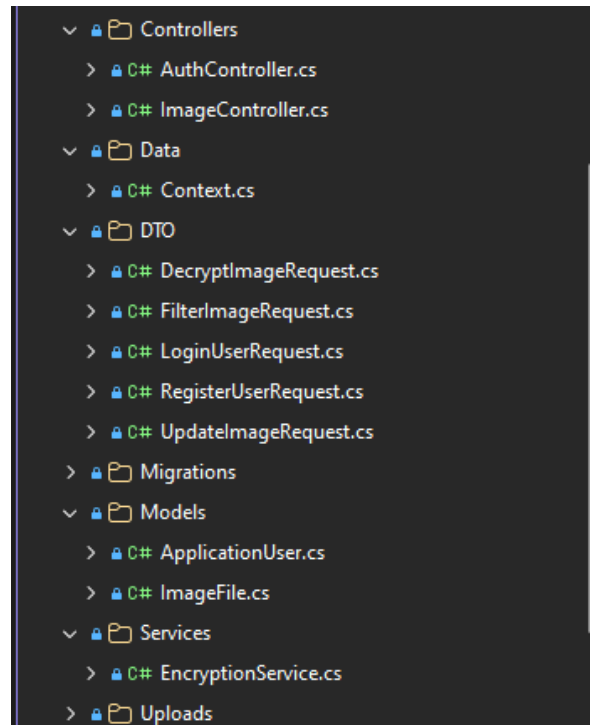
Serverul identifică utilizatorul, procesează imaginea și realizează criptarea acesteia. Fișierul criptat este salvat pe disc, iar metadatele asociate sunt introduse în SQL Server.

La vizualizare, procesul este realizat în sens invers. Clientul solicită imaginea și transmite parola necesară. Serverul identifică fișierul pe baza informațiilor din baza de date, îl decriptează și transmite conținutul rezultat către client pentru afișare.

5. Implementarea componentei server

Componenta server reprezintă nucleul aplicației **Photo Booth**, fiind responsabilă pentru procesarea cererilor provenite de la client, gestionarea utilizatorilor, accesul la baza de date și realizarea operațiilor asupra imaginilor.

Serverul a fost dezvoltat în C#, utilizând ASP.NET Core Web API. Arhitectura proiectului urmărește separarea responsabilităților între controllere, modele de date, obiecte utilizate pentru transferul datelor și serviciilor.



Această organizare permite separarea logicii aplicației în funcție de responsabilitatea fiecărei componente.

5.1 Configurarea aplicației – Program.cs

Fișierul Program.cs reprezintă punctul de configurare și pornire al aplicației ASP.NET Core.

În cadrul acestuia sunt înregistrate serviciile necesare funcționării serverului, printre care:

- Controllerele Web API;
- Entity Framework Core;
- Conexiunea cu SQL Server;
- ASP.NET Core Identity;
- Autentificarea JWT;
- Serviciul de criptare;
- Configurarea HTTPS;

- Mecanismele Dependency Injection

Pentru comunicarea cu baza de date Microsoft SQL Server este înregistrat contextul Entity Framework Core:

```
builder.Services.AddDbContext<Context>(options =>  
options.UseSqlServer(  
    builder.Configuration.GetConnectionString("DefaultConnection")));
```

Clasa *Context* reprezintă legătura dintre modelele aplicației și baza de date.

Connection string-ul este citit din configurația aplicației prin intermediul cheii *DefaultConnection*. Astfel, informațiile specifice serverului SQL nu sunt introduse direct în codul sursă al clasei Program.

Prin intermediul *UseSqlServer*, Entity Framework Core este configurat să utilizeze Microsoft SQL Server ca sistem de gestiune a bazei de date.

Pentru gestionarea utilizatorilor aplicației este utilizat ASP.NET Core Identity:

```
//Implementare Identity pentru autentificare si autorizare  
builder.Services  
    .AddIdentity<ApplicationUser, IdentityRole>()  
    .AddEntityFrameworkStores<Context>()  
    .AddDefaultTokenProviders();
```

ApplicationUser reprezintă modelul utilizatorului utilizat de aplicație, iar *IdentityRole* permite existența infrastructurii necesare gestionării rolurilor.

Metoda *.AddEntityFrameworkStores<Context>()* configurează Identity astfel încât informațiile despre utilizatori să fie persistente în baza de date gestionată prin *Context*.

AddDefaultTokenProviders() înregistrează furnizorii standard de token-uri utilizați de ASP.NET Core Identity.

În cadrul aplicației Photo Booth, Identity este utilizat în principal pentru înregistrarea utilizatorilor, autentificarea acestora și gestionarea securizată a parolelor.

Deoarece aplicația permite încărcarea imaginilor, au fost modificate limitele implicite pentru request-urile de tip multipart/form-data:

```
builder.Services.Configure<FormOptions>(options =>  
{  
    options.MultipartHeadersLengthLimit = 1_048_576;  
    options.MultipartBodyLengthLimit = 104_857_600;  
    options.ValueLengthLimit = 1_048_576;  
});
```

MultipartBodyLenghtLimit stabilește dimensiunea maximă permisă pentru corpul request-ului multipart. În configurația utilizată, limita este de aproximativ 100mb.

Această modificare permite aplicației să primească imagini de dimensiuni mai mari prin endpoint-ul de upload.

În timpul dezvoltării, comportamentul upload-ului a fost verificat și prin intermediul Postman.

Pentru protejarea endpoint-urilor API a fost implementată autentificarea bazată pe JSON Web Token (JWT)

Cheia necesară validării token-urilor este citită din configurația aplicației:

```
var jwtKey = builder.Configuration["Jwt:Key"];
```

Autentificarea este configurată astfel:

```
//Adaugare autentificare JWT
builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(options =>
{
    options.TokenValidationParameters =
        new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,

            ValidIssuer = builder.Configuration["Jwt:Issuer"],
            ValidAudience = builder.Configuration["Jwt:Audience"],

            IssuerSigningKey =
                new SymmetricSecurityKey(
                    Encoding.UTF8.GetBytes(jwtKey!)
                )
        };
});
```

La primirea unui JWT, serverul verifică emitentul, destinatarul, perioada de valabilitate și semnătura token-ului. Request-urile care necesită autentificarea pot astfel fi asociate utilizatorului care a efectuat login-ul.

Serviciul dezvoltat pentru criptarea și decriptarea imaginilor este înregistrat în sistemul de Dependency Injection:

```
builder.Services.AddScoped<EncryptionService>();
```


Prin această configurare, ASP.NET Core poate furniza automat o instanță *EncryptionService* componentelor care au nevoie de operațiile de criptare și decriptare, precum *ImageController*.

Serverul verifică la pornire existența directorului *Uploads*

```
var uploadsPath = Path.Combine(Directory.GetCurrentDirectory(), "Uploads");
if(!Directory.Exists(uploadsPath))
{
    Directory.CreateDirectory(uploadsPath);
}
app.UseStaticFiles(new StaticFileOptions
{
    FileProvider = new PhysicalFileProvider(uploadsPath),
    RequestPath = "/Uploads"
});
```

Dacă directorul nu există, acesta este creat automat. Fișierele imagine procesate de aplicație sunt stocate în acest director în formă criptată. *PhysicalFileProvider* îi spune aplicației din ce folder fizic de pe disc să citească/servească fișierele.

```
//Autorizare si autentificare pentru a proteja endpoint-urile API
app.UseAuthentication();
app.UseAuthorization();
```

În pipeline-ul aplicației au fost activate mecanismele de autentificare și autorizare. Acestea permit verificarea JWT-ului transmis de client înainte de executarea endpoint-urilor protejate și aplicarea regulilor definite prin atribute precum *[Authorize]*.

5.2 Configurarea contextului bazei de date

Pentru realizarea legăturii dintre aplicația ASP.NET Core și baza de date a fost creată clasa *Context* care extinde *IdentityDbContext<ApplicationUser>*

```
14 references
public class Context : IdentityDbContext<ApplicationUser>
{
    0 references
    public Context(DbContextOptions<Context> options) : base(options) { } //constructor DataBase
```

Moștenirea clasei menționate permite utilizarea aceluiași context atât pentru entitățile proprii aplicației, cât și pentru infrastructura ASP.NET Core Identity.

Astfel, baza de date poate conține atât informațiile necesare autentificării utilizatorilor, cât și datele specifice aplicației.

Obiectul *DbContextOptions<Context>* primit prin constructor conține configurația necesară conectării la baza de date. Această configurație este furnizată prin mecanismul de Dependency Injection configurat anterior în Program.cs

DbSet<ImageFile> reprezintă colecția entităților *ImageFile* gestionate de Entity Framework Core și corespunde tabelii utilizate pentru stocarea metadatelor imaginilor.

```
14 references
public DbSet <ImageFile> ImageFiles { get; set; } //tabela ImageFiles
```

Prin intermediul acestei proprietăți pot fi realizate operații asupra datelor.

Entity Framework Core realizează transformarea acestor operații C# în comenzile SQL necesare.

Un aspect important al clasei *Context* este faptul că aceasta nu moșteneste direct clasa standard *DbContext* ci *IdentityDbContext<ApplicationUser>*.

Această alegere permite integrarea modelelor aplicației cu sistemul ASP.NET Core Identity.

Prin urmare, același context gestionează două categorii principale de informații: ASP.NET Core Identity, *ImageFiles*;

Metoda *OnModelCreating* definește inițial și câteva înregistrări de test. Metoda *HasData()* reprezintă mecanismul oferit de Entity Framework Core și permite introducerea unor date inițiale odată cu configurarea modelului bazei de date.

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);

    modelBuilder.Entity<ImageFile>().HasData(
        new ImageFile { Id = 1, Subject = "Sample Image 1", FilePath = "path/to/image1.jpg" },
        new ImageFile { Id = 2, Subject = "Sample Image 2", FilePath = "path/to/image2.jpg" },
        new ImageFile { Id = 3, Subject = "Sample Image 3", FilePath = "path/to/image3.jpg" }
    );
}
```

Acest test a rămas din fazele incipiente ale proiectului și nu mai reprezintă viziunea finală asupra unui element situat în baza de date.

5.3 Modelul Imaginilor

Pentru reprezentarea imaginilor gestionate de aplicație a fost creată clasa *ImageFile*. Aceasta nu conține datele binare ale imaginii, ci metadatele necesare identificării, organizării, asocierii cu un utilizator și decriptării fișierului.

```

namespace Photo_Booth_Server_API.Models
{
    16 references
    public class ImageFile
    {
        10 references
        public int Id { get; set; }
        11 references
        public string Subject { get; set; } = string.Empty;

        10 references
        public string FilePath { get; set; } = string.Empty;

        12 references
        public string UserId { get; set; } = string.Empty; // Cheie externa pentru ApplicationUser

        //Content Type generic pentru acceptarea mai multor tipuri de fisiere (ex: image/jpeg, image/png, etc.)
        6 references
        public string ContentType { get; set; } = string.Empty;

        //Proprietati pentru criptare

        6 references
        public string Salt { get; set; } = string.Empty;
        6 references
        public string Nonce { get; set; } = string.Empty;
        6 references
        public string Tag { get; set; } = string.Empty;
    }
}

```

Fiecare instanță ImageFile corespunde unei înregistrări din tabela ImageFiles gestionată prin Entity Framework Core.

Proprietățile Id, Subject, FilePath reprezintă datele de bază ale unei imagini. UserId este folosit pentru asocierea unei imagini cu un utilizator. String ContentType păstrează extensia imaginii oferite.

Salt, Nonce, Tag reprezintă metadatele necesare criptării.

Salt este utilizat în procesul de derivare a cheii criptografice din parola introdusă de utilizator. Utilizarea unui salt contribuie la obținerea unei chei derivate specifice procesului de criptare.

Nonce reprezintă valoarea utilizată de algoritmul AES-GCM pentru operația de criptare și este necesară ulterior pentru decriptare.

Tag reprezintă tag-ul de autentificare generat de AES-GCM. Acesta este utilizat la decriptare pentru verificarea integrității și autenticității datelor criptate.

Modelul ImageFile realizează legătura dintre cele trei zone principale ale aplicației. Baza de date nu trebuie să conțină imaginea propriu-zisă, ci informațiile necesare pentru ca serverul să poată identifica, autoriza, organiza și decripta fișierul corespunzător.

5.4 Data Transfer Object (DTO)

Pentru transmiterea datelor între aplicația client și API au fost create clase dedicate de tip DTO (Data Transfer Object). Aceste clase conțin doar informațiile necesare unei anumite operații și permit separarea datelor primite de API de modelele utilizate pentru baza de date.

Pentru transmiterea credențialelor necesare autentificării a fost creată clasa LoginUserRequest.

```
1 namespace Photo_Booth_Server_API.DTO
2 {
3     1 reference
4     public class LoginUserRequest
5     {
6         1 reference
7         public string Email { get; set; } = string.Empty;
8         1 reference
9         public string Password { get; set; } = string.Empty;
10    }
11 }
```

Obiectul este transmis către endpoint-ul de autentificare sub formă de JSON.

```
Photo_Booth_Server_API.DTO
namespace Photo_Booth_Server_API.DTO
{
    1 reference
    public class RegisterUserRequest
    {
        1 reference
        public string Username { get; set; }
        2 references
        public string Email { get; set; }
        1 reference
        public string Password { get; set; }
    }
}
```

RegisterUserRequest transmite un JSON către endpoint-ul de înregistrare al serverului.

```

namespace Photo_Booth_Server_API.DTO
{
    1 reference
    public class UpdateImageRequest
    {
        1 reference
        public string Subject { get; set; } = string.Empty;
    }
}

```

Separarea acestui DTO de modelul ImageFile permite clientului să transmită doar informația care trebuie modificată, fără a trimite întreaga entitate și fără a putea modifica direct alte proprietăți precum UserId, FilePath, Salt, Nonce sau Tag.

```

namespace Photo_Booth_Server_API.DTO
{
    1 reference
    public class FilterImageRequest
    {
        2 references
        public string Filter { get; set; } = string.Empty;
        2 references
        public string EncryptionPassword { get; set; } = string.Empty;
    }
}

```

Parola este necesară deoarece imaginea existentă este stocată pe server în formă criptată. Pentru aplicarea unui filtru, serverul trebuie mai întâi să decripteze imaginea, să efectueze procesarea și apoi să creeze rezultatul înainte de salvare.

```

namespace Photo_Booth_Server_API.DTO
{
    1 reference
    public class DecryptImageRequest
    {
        1 reference
        public string EncryptionPassword { get; set; } = string.Empty;
    }
}

```

Imaginea nu este stocată pe server în format direct vizualizabil, ci sub formă criptată. Din acest motiv, atunci când utilizatorul dorește să vizualizeze o imagine, serverul trebuie să primească parola cu care aceasta a fost criptată.

5.5 Serviciul de criptare și decriptare

Pentru protejarea imaginilor stocate pe server a fost implementată clasa *EncryptionService*. Aceasta este responsabilă pentru criptarea datelor înainte de salvarea pe disc și pentru decriptarea acestora atunci când utilizatorul solicită vizualizarea imaginii.

Serviciul utilizează algoritmul AES-GCM, iar cheia de criptare nu este stocată direct. Aceasta este derivată din parola introdusă de utilizator folosind PBKDF2.

Metoda *Encrypt* primește conținutul imaginii sub forma unui vector de bytes și parola introdusă de utilizator.

Pentru fiecare operație de criptare este generat un Salt aleator de 16 bytes. Acesta este utilizat în procesul de derivare a cheii și permite ca aceeași parolă să genereze chei diferite atunci când sunt folosite valori Salt diferite.

Este generat și un Nonce aleator de 12 bytes, utilizat de AES-GCM pentru operația de criptare.

Tag are o dimensiune de 16 bytes și este generat în cadrul operației AES-GCM. Acesta permite verificarea autenticității datelor în timpul decriptării.

```
{
    2 references
    public byte[] Encrypt(byte[] data, string password, out byte[] salt, out byte[] nonce, out byte[] tag)
    {
        salt = RandomNumberGenerator.GetBytes(16);
        nonce = RandomNumberGenerator.GetBytes(12);
        tag = new byte[16];
    }
}
```

Parola utilizatorului nu este utilizată direct drept cheie AES. În schimb, cheia este derivată prin intermediul clasei *Rfc289DeriveBytes* care implementează PBKDF2.

```
using var deriveBytes = new Rfc289DeriveBytes(password, salt, 100000, HashAlgorithmName.SHA256);
var key = deriveBytes.GetBytes(32);
```

Cele 100.000 de iterații fac derivarea cheii mai costisitoare din punct de vedere computațional decât utilizarea directă a parolei. Rezultatul are 32 bytes (256 biți) și este utilizat drept cheia AES.

După obținerea cheii, este creat bufferul în care va fi stocată imaginea criptată.

```
var encrypted = new byte[data.Length];
using var aes = new AesGcm(key, 16);

aes.Encrypt(nonce, data, encrypted, tag);
return encrypted;
```

AES-GCM realizează atât criptarea datelor, cât și generarea unui tag de autentificare. Metoda returnează conținutul criptat.

Metoda Decrypt funcționează exact la fel, cu mențiunea că va returna un rezultat decriptat în locul celui criptat.

5.6 Controller-ul de autentificare AuthController

Clasa AuthController gestionează operațiunile de înregistrare și autentificare a utilizatorilor. Controller-ul utilizează ASP.NET Core Identity pentru gestionarea conturilor și generează token-uri JWT (JSON Web Token) pentru utilizatorii autentificați cu succes.

```
[Route("api/[controller]")]
[ApiController]
1 reference
public class AuthController : ControllerBase
{
```

Prin ruta definită, controller-ul este accesibil prin /api/auth, peste care sunt adăugate rutele specifice operațiilor de register și login.

Controller-ul primește prin Dependency Injection două componente:

```
private readonly UserManager<ApplicationUser> _userManager;
private readonly IConfiguration _configuration;

0 references
public AuthController(UserManager<ApplicationUser> userManager, IConfiguration configuration)
{
    _userManager = userManager;
    _configuration = configuration;
}
```

UserManager este componenta ASP.NET Core Identity prin care sunt realizate operațiunile asupra utilizatorilor, precum căutarea unui cont, crearea acestuia și verificarea parolei.

IConfiguration este utilizat pentru accesarea configurației JWT definite pentru aplicație, în special Jwt:Key, Jwt:Issuer și Jwt:Audience.

Înregistrarea unui utilizator este realizată prin endpoint-ul

```
[HttpPost("register")]
0 references
public async Task<IActionResult> Register(RegisterUserRequest request)
```

Acesta corespunde unui request de tipul: POST / api / auth/ register.

Datele necesare sunt primite prin DTO-ul RegisterUserRequest. Primul pas constă în verificarea existenței unui cont cu aceeași adresă de e-mail:

```
var existingUser = await _userManager.FindByEmailAsync(request.Email);
if(existingUser != null)
{
    return BadRequest("Email already registered");
}

var user = new ApplicationUser
{
    UserName = request.Username,
    Email = request.Email
};
```

Dacă adresa este deja utilizată, serverul respinge înregistrarea, altfel este construit un nou ApplicationUser.

Crearea efectivă a contului este delegată către ASP.NET Core Identity:

```
var result = await _userManager.CreateAsync(
    user,
    request.Password
);

if(!result.Succeeded)
{
    return BadRequest(result.Errors);
}

return Ok("User Registered successfully");
```


Prin urmare, controller-ul nu introduce manual parola într-un câmp al bazei de date. Parola este transmisă către Identity, care gestionează procesul de creare a utilizatorului și stocarea securizată a credențialelor. Dacă Identity raportează o eroare, aceasta este returnată clientului.

Autentificarea utilizatorului este realizată prin:

```
[HttpPost("login")]
0 references
public async Task<IActionResult> Login(LoginUserRequest request)
{
```

Care corespunde rutei POST / api / auth / login.

Mai întâi este căutat utilizatorul pe baza adresei de email, dacă utilizatorul există, parola este verificată folosind UserManager. Atât inexistența utilizatorului, cât și introducerea unei parole incorecte determină returnarea mesajului *Invalid Credentials*

Dacă datele sunt valide este generat token-ul JWT.

```
var user = await _userManager.FindByEmailAsync(request.Email);
if (user == null)
{
    return Unauthorized("Invalid credentials");
}

var isPasswordValid = await _userManager.CheckPasswordAsync(user, request.Password);
if (!isPasswordValid)
{
    return Unauthorized("Invalid credentials");
}

var token = GenerateJwtToken(user);
return Ok(new { Token = token });
```

Pentru generarea token-ului JWT sunt definite informațiile care vor fi introduse în token sub formă de claims. Token-ul conține identificatorul unic al utilizatorului, numele utilizatorului și adresa de email.

Identificatorul unic este deosebit de important pentru aplicație deoarece permite identificarea utilizatorului autentificat în operațiile asupra imaginilor. Cheia configurată pentru JWT este apoi transformată în cheie simetrică.

```

1 reference
private string GenerateJwtToken(ApplicationUser user)
{
    var claims = new[]
    {
        new Claim(ClaimTypes.NameIdentifier, user.Id),
        new Claim(ClaimTypes.Name, user.UserName!),
        new Claim(ClaimTypes.Email, user.Email!)
    };
    var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_configuration["Jwt:Key"]!));
    var credentials = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
    var token = new JwtSecurityToken(
        _configuration["Jwt:Issuer"],
        _configuration["Jwt:Audience"],
        claims,
        expires: DateTime.UtcNow.AddHours(1),
        signingCredentials: credentials
    );
    return new JwtSecurityTokenHandler().WriteToken(token);
}

```

Pentru semnarea token-ului este utilizat algoritmul HMAC-SHA256. Token-ul este apoi construit folosind Issuer, Audience, claim-urile definite anterior și perioada de valabilitate de o oră. În final, acesta este serializat într-un șir de caractere care poate fi transmis clientului.

5.7 Controller-ul pentru gestionarea imaginilor – Image Controller

ImageController reprezintă componenta principală a API-ului pentru gestionarea imaginilor. Acesta implementează toate operațiile cerute.

Controller-ul utilizează două dependențe principale. Context permite accesul la baza de date prin Entity Framework Core, iar EncryptionService furnizează operațiile de criptare și decriptare prezentate anterior.

```

private readonly Context _context;
private readonly EncryptionService _encryptionService;

0 references
public ImageController(Context context, EncryptionService encryptionService)
{
    _context = context;
    _encryptionService = encryptionService;
}

```

Toate endpoint-urile din ImageController sunt protejate prin autentificare JWT. Prin această configurare, operațiile asupra imaginilor necesită existența unui token JWT valid.

```

[Route("api/[controller]")]
[ApiController]
[Authorize(AuthenticationSchemes = JwtBearerDefaults.AuthenticationScheme)]

```

Endpoint-ul `Get/api/image` returneaza imaginile asociate utilizatorului autentificat. Interogarea filtrează înregistrările din `ImageFiles` pe baza identificatorului extras din JWT. Este implementată și recuperarea unei singure imagini prin identificatorul: `GET/api/image/{id}`

```
[HttpGet]
0 references
public async Task <ActionResult<List<ImageFile>>> GetImageFile() //Intoarce toate imaginile din lista
{
    var userId = User.FindFirstValue(ClaimTypes.NameIdentifier);

    if (userId == null)
    {
        return Unauthorized();
    }

    var images = await _context.ImageFiles
        .Where(x => x.UserId == userId)
        .ToListAsync();

    return Ok(images);
}
```

Pentru actualizarea subiectului a fost implementat endpoint-ul: `PUT/api/image/{id}`. Acesta utilizează DTO-ul `UpdateImageRequest`. Astfel, clientul poate modifica subiectul fără a putea modifica prin același request informațiile sensibile sau interne ale imaginii.

```
[HttpPut("{id}")]
0 references
public async Task<ActionResult> UpdateImage(int id, [FromBody] UpdateImageRequest request)
{
    var userId = User.FindFirstValue(ClaimTypes.NameIdentifier);
    if(userId == null)
    {
        return Unauthorized();
    }

    var existingImage = await _context.ImageFiles
        .FirstOrDefaultAsync(x =>
            x.Id == id &&
            x.UserId == userId);
    if (existingImage == null)
    {
        return NotFound();
    }

    existingImage.Subject = request.Subject;
    await _context.SaveChangesAsync();
    return NoContent();
}
```

Upload-ul reprezintă una dintre funcționalitățile principale ale controller-ului și este realizat prin: POST /api/image/upload.

Endpoint-ul primește trei informații prin multipart/form-data. Acestea reprezintă fișierul selectat, subiectul definit de utilizator și parola utilizată pentru criptare.

```
var uniqueFileName = Guid.NewGuid().ToString() + Path.GetExtension(file.FileName);
```

Pentru evitarea conflictelor între numele fișierelor este generat un identificator GUID. Fișierul va fi plasat în directorul Uploads.

Fișierul primit este mai întâi convertit într-un vector de bytes, datele sunt apoi transmise către EncryptionService. Pe disc este scris rezultatul criptat. Prin urmare, imaginea originală nu este salvată în clar în directorul Uploads.

```
using var memoryStream = new MemoryStream();
await file.CopyToAsync(memoryStream);

var imageBytes = memoryStream.ToArray();

var encryptedBytes = _encryptionService.Encrypt(imageBytes, encryptionPassword, out var salt, out var nonce, out var tag);

await System.IO.File.WriteAllBytesAsync(filePath, encryptedBytes);
```

După criptare este creată înregistrarea ImageFile. Salt, Nonce, Tag sunt convertite în Base64 pentru a putea fi stocate ca șiruri de caractere în baza de date. Astfel, upload-ul produce două rezultate distincte: fișierul criptat pe disc și metadatele în SQL Server.

```
//Creare obiect ImageFile pentru a stoca metadatele in baza de date
var image = new ImageFile
{
    Subject = subject,
    FilePath = $"/Uploads/{uniqueFileName}",
    UserId = userId,
    ContentType = file.ContentType,
    Salt = Convert.ToBase64String(salt),
    Nonce = Convert.ToBase64String(nonce),
    Tag = Convert.ToBase64String(tag)
};

_context.ImageFiles.Add(image);
await _context.SaveChangesAsync();
return Ok(image);
```

Duplicarea este realizată prin POST /api/image/{id}/duplicate. Serverul verifică mai întâi că imaginea solicitată aparține utilizatorului autentificat și că fișierul fizic există. Este generat apoi un nou nume. Deoarece sursa este deja criptată, duplicarea copiază direct fișierul criptat. Este apoi creată o nouă înregistrare ImageFile. Deoarece conținutul criptat este copiat fără a fi recriptat, copia utilizează aceleași valori criptografice ca originalul. Noua imagine primește însă propriul Id și propriul FilePath

```
var newFileName = Guid.NewGuid() + extension;
var destinationPath = Path.Combine(uploadsPath, newFileName);

System.IO.File.Copy(sourcePath, destinationPath);

var duplicatedImage = new ImageFile
{
    Subject = image.Subject,
    FilePath = $"/Uploads/{newFileName}",
    UserId = userId,

    ContentType = image.ContentType,
    Salt = image.Salt,
    Nonce = image.Nonce,
    Tag = image.Tag
};

_context.ImageFiles.Add(duplicatedImage);
await _context.SaveChangesAsync();

return Ok(duplicatedImage);
```

Aplicarea filtrelor este realizată prin POST /api/image/{id}/filter. Operația este cea mai complexă deoarece imaginea stocată pe server este criptată. Mai întâi sunt recuperate datele criptate și metadatele. Imaginea este apoi decriptată folosind parola transmisă prin FilterImageRequest.

```
try
{
    //Citire fisier criptat
    var encryptedBytes = await System.IO.File.ReadAllBytesAsync(sourcePath);

    //Decriptare imaginea originala folosind parola si metadatele salvate in baza de date
    var salt = Convert.FromBase64String(imageFile.Salt);
    var nonce = Convert.FromBase64String(imageFile.Nonce);
    var tag = Convert.FromBase64String(imageFile.Tag);

    //Decriptare Imagine Originala
    var decryptedBytes = _encryptionService.Decrypt(encryptedBytes, request.EncryptionPassword, salt, nonce, tag);
```

Imaginea decriptată este încărcată în MagickImage, iar filtrul este selectat în funcție de valoarea primită. Au fost implementate patru filtre: grayscale, sepia, blue și negate.

```
//Incarcare imagine direct din memorie
using var image = new MagickImage(decryptedBytes);

//Selectare filtre din MagickImage
switch (request.Filter.ToLower())
{
    case "grayscale":
        image.Grayscale();
        break;
    case "sepia":
        image.SepiaTone();
        break;
    case "blur":
        image.Blur(0, 5);
        break;
    case "negate":
        image.Negate();
        break;
    default:
        return BadRequest("Please specify a valid filter: grayscale, sepia, blur, negate.");
}
```

Rezultatul este convertit din nou în bytes și recriptat cu metadata criptografice noi. Este apoi creat un fișier nou și o nouă înregistrare în baza de date. Subiectul rezultatului este construit din subiectul original și filtrul aplicat. Prin urmare, implementarea respectă cerința conform căreia filtrarea trebuie să genereze o imagine nouă fără modificarea originalului. Dacă parola furnizată este incorectă și decriptarea eșuează, excepția criptografică este tratată.

```
//Transformarea imaginii inapoi in bytes
var filteredBytes = image.ToArray();

//Criptare rezultatul cu metadata noua

var encryptedFilteredBytes = _encryptionService.Encrypt(filteredBytes, request.EncryptionPassword, out var newSalt, out var newNonce, out var newTag);

//Salvare fisier criptat nou
var newFileName = Guid.NewGuid().ToString() + Path.GetExtension(sourcePath);
var destinationPath = Path.Combine(Directory.GetCurrentDirectory(), "Uploads", newFileName);
await System.IO.File.WriteAllBytesAsync(destinationPath, encryptedFilteredBytes);

//Creare obiect ImageFile pentru a stoca metadatale in baza de date
var filteredImage = new ImageFile
{
    Subject = imageFile.Subject + " - " + request.Filter,
    FilePath = $"Uploads/{newFileName}",
    UserId = userId,
    ContentType = imageFile.ContentType,
    Salt = Convert.ToBase64String(newSalt),
    Nonce = Convert.ToBase64String(newNonce),
    Tag = Convert.ToBase64String(newTag)
};

_context.ImageFiles.Add(filteredImage);
await _context.SaveChangesAsync();
return Ok(filteredImage);
}
```

Pentru vizualizare este disponibil endpoint-ul POST /api/image/{id}/decrypt. Serverul verifică mai întâi că imaginea aparține utilizatorului autentificat și localizează fișierul criptat. Metadatale stocate în Base64 sunt convertite înapoi în byte[]

Imaginea este decriptată. Spre deosebire de upload, serverul nu salvează rezultatul decriptat pe disc. Acesta este returnat direct clientului.

```
var salt = Convert.FromBase64String(imageFile.Salt);
var nonce = Convert.FromBase64String(imageFile.Nonce);
var tag = Convert.FromBase64String(imageFile.Tag);

try
{
    //se incearca decriptarea imaginii folosind parola si metadatele salvate in baza de date
    var decryptedBytes = _encryptionService.Decrypt(encryptedBytes, request.EncryptionPassword, salt, nonce, tag);
    return File(decryptedBytes, imageFile.ContentType);
}

catch (CryptographicException)
{
    return BadRequest("Parola gresita");
}
```

Ștergerea unei imagini este implementată prin DELETE /api/image/{id}. Serverul verifică dacă înregistrarea există și aparține utilizatorului autentificat. Ștergerea este realizată atât asupra fișierului fizic cât și asupra înregistrării din baza de date. Astfel, ștergerea nu lasă fișierele criptate fără corespondent.

```
var physicalPath = Path.Combine(
    Directory.GetCurrentDirectory(),
    existingImage.FilePath.TrimStart('/')
);

if (System.IO.File.Exists(physicalPath))
{
    System.IO.File.Delete(physicalPath);
}

_context.ImageFiles.Remove(existingImage);
await _context.SaveChangesAsync();

return NoContent();
```

Organizarea imaginilor pe subiecte este realizată prin implementarea a două endpoint-uri. Primul permite căutarea imaginilor după subiect: GET /api/image/subject/{subject}.

Interogarea verifică atât utilizatorul cât și subiectul. Astfel rezultatele sunt limitate la imaginile utilizatorului autentificat.

Al doilea endpoint este destinat obținerii subiectelor disponibile: GET /api/image/subject.

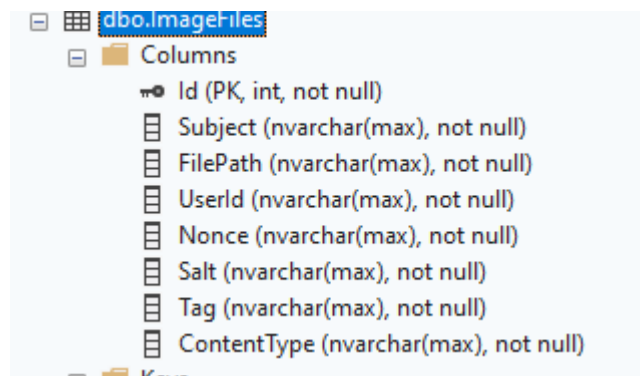
Lista este construită prin selectarea proprietății Subject și eliminarea duplicatelor.

```
var subject = await _context.ImageFiles
    .Where(x => x.UserId == userId)
    .Select(x => x.Subject)
    .Distinct()
    .ToListAsync();
```

6. Baza de date și operațiile CRUD

Pentru asigurarea persistenței datelor, aplicația Photo Booth utilizează Microsoft SQL Server, iar accesul la baza de date este realizat prin intermediul Entity Framework Core.

Baza de date are două roluri principale: gestionarea informațiilor asociate utilizatorilor prin ASP.NET Core Identity și stocarea metadatelor asociate imaginilor. Fișierele imagine propriu-zise nu sunt păstrate în baza de date, ci separat în directorul Uploads, în formă criptată.



Una dintre cerințele proiectului presupune realizarea operațiilor CRUD, aceste operații au fost implementate în ImageController.

Operația Create este realizată atunci când utilizatorul încarcă o imagine. După criptarea fișierului este construit un obiect ImageFile care conține metadatele imaginii.

```
//Creare obiect ImageFile pentru a stoca metadatele in baza de date
var image = new ImageFile
{
    Subject = subject,
    FilePath = $"/Uploads/{uniqueFileName}",
    UserId = userId,
    ContentType = file.ContentType,
    Salt = Convert.ToBase64String(salt),
    Nonce = Convert.ToBase64String(nonce),
    Tag = Convert.ToBase64String(tag)
};
```

Operațiile de tip Create sunt realizate și la duplicarea unei imagini și la aplicarea unui filtru, deoarece ambele funcționalități generează o nouă înregistrare ImageFile.

Operația de tip Read este utilizată pentru recuperarea informațiilor existente în baza de date. Pentru obținerea imaginilor utilizatorului este utilizată interogarea

```
var image = await _context.ImageFiles
    .FirstOrDefaultAsync(x =>
        x.Id == id &&
        x.UserId == userId);
```

Operațiile de citire sunt utilizate și pentru căutarea imaginilor după subiect și pentru obținerea listei subiectelor asociate utilizatorului.

Operația Update este implementată pentru modificarea subiectului asociat unei imagini. Serverul identifică mai întâi imaginea pe baza Id și UserId, după care actualizează proprietatea.

```
existingImage.Subject = request.Subject;
await _context.SaveChangesAsync();
```

Utilizarea DTO-ului UpdateImageRequest permite modificarea exclusivă a proprietății Subject, fără expunerea celorlalte proprietăți ale entității către operația de actualizare.

Operația Delete elimină atât înregistrarea din baza de date cât și fișierul criptat corespunzător. Înregistrarea este eliminată prin:

```
_context.ImageFiles.Remove(existingImage);  
await _context.SaveChangesAsync;
```

Un aspect important al operațiilor asupra bazei de date este utilizarea identificatorului utilizatorului autentificat. Identificatorul este recuperat din JWT și este inclus în interogările asupra imaginilor.

Pe lângă tabela ImageFiles, baza de date conține și structurile generate de ASP.NET Core Identity ceea ce permite utilizarea aceleiași baze de date pentru persistența conturilor și pentru datele specifice Photo Booth.

Structura bazei de date este gestionată cu ajutorul Entity Framework Core Migrations. Migrations permit păstrarea modificărilor efectuate asupra modelelor și aplicarea acestora asupra unei baze de date. Acest mecanism a fost utilizat inclusiv pentru reconstruirea bazei de date pe un alt sistem de dezvoltare, fără transferarea manuala a bazei de date existente.

Aplicarea migrațiilor se poate realiza prin: dotnet ef database update.

7. Implementarea aplicației client Unity

Componenta client a aplicației Photo Booth a fost dezvoltată utilizând Unity și limbajul de programare C#. Aceasta reprezintă interfața prin care utilizatorul interacționează cu sistemul și comunică cu componenta server prin intermediul API-ului REST.

Clientul este responsabil pentru colectarea datelor introduse de utilizator, transmiterea request-urilor către server și prezentarea rezultatelor primite. Operațiile sensibile precum autentificarea, accesul la baza de date, criptarea, decriptarea și procesarea imaginilor sunt realizate la nivelul serverului.

Interfața aplicației este împărțită în două zone principale. Prima este destinată înregistrării și autentificării utilizatorului, unde acesta poate crea un cont sau se poate autentifica folosind datele unui cont existent.

Dupa autentificare, utilizatorul are acces la galeria de imagini, unde poate încărca și gestiona imaginile asociate contului său.

Interfața Unity nu accesează direct SQL Server și nici fișierele din directorul Uploads. Toate aceste operații sunt intermediare de API.

7.1 Configurarea comunicației cu serverul – ApiClient

Pentru centralizarea informațiilor necesare comunicării dintre clientul Unity și componenta server a fost creată clasa statică ApiClient.

Fiind declarată static, clasa poate fi accesată de celelalte scripturi Unity fără a fi necesară crearea unei instanțe sau atașarea acesteia unui GameObject.

```
16 references
public static class ApiClient
{
    public const string BaseUrl = "https://localhost:7043/api";
    public static string Token = "";
}
```

Deoarece Token este static, valoarea poate fi accesată de diferitele componente ale clientului pe durata executării aplicației. În implementarea actuală, token-ul este păstrat în memorie, nu într-un fișier local. Prin urmare, la închiderea aplicației valoarea este pierdută, iar utilizatorul trebuie să se autentifice din nou la următoarea pornire.

7.2 Serviciul de autentificare – AuthService

Clasa AuthService este responsabilă pentru comunicarea dintre interfața Unity și endpoint-urile de autentificare ale serverului. Aceasta implementează operațiile de login și registre, folosind UnityWebRequest pentru transmiterea datelor către ASP.NET Core Web API.

Clasa moștenește MonoBehaviour deoarece metodele de comunicare cu serverul sunt implementate sub forma unor corutine Unity.

Utilizarea corutinelor permite executarea asincronă a request-urilor fără blocarea firului principal al aplicației Unity.

Pentru autentificarea utilizatorului metoda Login primește adresa de email, parola utilizatorului și un callback utilizat pentru informarea interfeței dacă autentificarea a reușit. Mai întâi este construit un obiect LoginRequest, acesta este convertit în format JSON, iar rezultatul este transmis către server.

```
© Unity Script (1 asset reference) 2 references
public class AuthService : MonoBehaviour
{
    1 reference
    public IEnumerator Login(string email, string password, System.Action<bool> callback)
    {
        var loginData = new LoginRequest
        {
            email = email,
            password = password
        };

        string json = JsonUtility.ToJson(loginData);
    }
}
```

Pentru autentificare este creat un request POST către `"/auth/login"`.

JSON-ul este convertit într-un vector de bytes și atașat request-ului specificând tipul datelor transmise.

```
byte[] body = Encoding.UTF8.GetBytes(json);

request.uploadHandler = new UploadHandlerRaw(body);
request.downloadHandler = new DownloadHandlerBuffer();

request.SetRequestHeader("Content-Type", "application/json");

yield return request.SendWebRequest();
```

Dacă serverul returnează un răspuns de succes, conținutul răspunsului este deserializat într-un obiect `LoginResponse`. Token-ul JWT primit de la server este apoi stocat. Astfel token-ul poate fi reutilizat de celelalte servicii client pentru accesarea endpoint-urilor protejate. În funcție de rezultatul operației, metoda apelează callback-ul cu `true` sau `false`, iar interfața poate decide dacă trebuie să permită accesul la galerie sau să afișeze un mesaj de eroare.

```
yield return request.SendWebRequest();

if (request.result == UnityWebRequest.Result.Success)
{
    var response = JsonUtility.FromJson<LoginResponse>(request.downloadHandler.text);

    ApiClient.Token = response.token;

    Debug.Log("Login Successful");
    Debug.Log("Token" + ApiClient.Token);

    callback?.Invoke(true);
}
else
{
    Debug.LogError("Login failed: " + request.responseCode + " " + request.downloadHandler.text);
}

callback?.Invoke(false);
```

Înregistrarea este implementată prin metoda `Register()`, datele sunt introduse într-un obiect `RegisterRequest`, acesta este serializat în JSON, iar request-ul este transmis către POST `/api/auth/register`.

```

var registerData = new RegisterRequest
{
    username = username,
    email = email,
    password = password
};

string json = JsonUtility.ToJson(registerData);

using var request = new UnityWebRequest(ApiClient.BaseUrl + "/auth/register", "POST");

byte[] body = Encoding.UTF8.GetBytes(json);

request.uploadHandler = new UploadHandlerRaw(body);
request.downloadHandler = new DownloadHandlerBuffer();
request.SetRequestHeader("Content-Type", "application/json");
yield return request.SendWebRequest();

```

După primirea răspunsului, codul verifică status code-ul HTTP. Orice răspuns din intervalul 2xx este considerat success. Clientul poate astfel actualiza interfața fără ca AuthService să fie responsabil direct pentru afișarea mesajelor.

Un aspect deosebit de important al implementării este utilizarea callback-urilor. AuthService nu schimbă direct scenele și nu afișează elementele UI. El returnează doar rezultatul operației.

7.3 Interfața de autentificare – LoginUI

Clasa LoginUI gestionează interacțiunea utilizatorului cu interfața de autentificare. Aceasta preia datele introduse în câmpurile de email și parolă, apelează AuthService și actualizează interfața în funcție de rezultatul autentificării.

```

public class LoginUI : MonoBehaviour
{
    [SerializeField]
    private TMP_InputField emailInput;
    [SerializeField]
    private TMP_InputField passwordInput;
    [SerializeField]
    private TMP_Text errorText;
    [SerializeField]
    private AuthService authService;
    [SerializeField]
    private Canvas registerUI;
    [SerializeField]
    private Canvas loginUI;
}

```

Atributul [SerializeField] permite asocierea acestor câmpuri cu obiecte corespunzătoare din scena Unity prin intermediul interfeței grafice a editorului, păstrând în același timp câmpurile private.

Pentru introducerea datelor sunt utilizate componente TMP_InputField, iar error text este utilizat pentru afișarea rezultatului unei autentificări nereușite.

Interfața permite trecerea de la formularul de autentificare la cel de înregistrare prin activarea și dezactivarea celor două obiecte Canvas.

Astfel, cele două formulare pot exista în aceeași scenă Unity, fiind afișat doar cel necesar la momentul respectiv.

```
0 references
public void ShowRegister()
{
    loginUI.gameObject.SetActive(false);
    registerUI.gameObject.SetActive(true);
}

public void ShowLogin()
{
    registerUI.gameObject.SetActive(false);
    loginUI.gameObject.SetActive(true);
}
```

Atunci când utilizatorul apasă butonul de Login este apelată metoda Login(). Valorile introduse în cele două câmpuri sunt transmise către metoda Login() din AuthService. Deoarece comunicarea cu API-ul este realizată printr-o corutină aceasta este pornită folosind StartCoroutine().

Metoda OnLoginFinished este transmisă drept callback și va fi executată după finalizarea request-ului. Mesajul de eroare este ascuns, iar utilizatorul este transferat către scena principală a aplicației: Gallery.

```

0 references
public void Login()
{
    errorText.text = "";

    StartCoroutine(authService.Login(emailInput.text, passwordInput.text, OnLoginFinished));
}

1 reference
private void OnLoginFinished(bool success)
{
    if (success)
    {
        errorText.gameObject.SetActive(false);
        Debug.Log("Logat cu succes!");
        SceneManager.LoadScene("Gallery");
    }
    else
    {
        errorText.gameObject.SetActive(true);
        errorText.text = "Email sau parola incorecta!";
    }
}

```

7.4 Interfața de înregistrare – RegisterUI

Clasa RegisterUI gestionează interfața utilizată pentru crearea unui cont nou. Rolul acesteia este de a prelua datele introduse de utilizator, de a transmite informațiile către AuthService și de a afișa rezultatul operației de înregistrare.

Pentru preluarea datelor și afișarea mesajelor sunt definite următoarele componente. Cele trei componente TMP_InputField permit introducerea numelui de utilizator, adresei de email și parolei, iar messageText este utilizat pentru comunicarea rezultatului operației.

Referința către AuthService permite separarea interfeței de logica efectivă de comunicare cu API-ul.

```

public class RegisterUI : MonoBehaviour
{
    [SerializeField]
    private TMP_InputField usernameInput;
    [SerializeField]
    private TMP_InputField emailInput;
    [SerializeField]
    private TMP_InputField passwordInput;
    [SerializeField]
    private TMP_Text messageText;
    [SerializeField]
    private AuthService authService;
}

```

0 references

La apăsarea butonului de înregistrare este executată metoda Register(). Datele introduse sunt preluate direct din componentele UI și transmise metodei Register() din AuthService.

Operația este executată sub forma unei corutine deoarece presupune transmiterea unui request către componenta server.

```
0 references
public void Register()
{
    messageText.text = "";
    StartCoroutine(authService.Register(usernameInput.text, emailInput.text, passwordInput.text, OnRegisterFinished));
}
```

După finalizarea request-ului, AuthService transmite rezultatul către OnRegisterFinished(). În funcție de valoarea primită, interfața afișează un mesaj corespunzător. În cazul unei erori utilizatorul este informat că procesul de crearea a contului nu a fost finalizat.

7.5 Serviciul pentru gestionarea imaginilor – ImageService

Clasa ImageService centralizează comunicația dintre clientul Unity și endpoint-urile API responsabile de gestionarea imaginilor. Prin intermediu lăcestei clase sunt realizate operațiile de recuperare a imaginilor, decriptare, duplicare, ștergere, aplicare a filtrelor și încărcare.

Clasa moștenește MonoBehaviour, deoarece metodele sale sunt implementate drept corutine Unity.

Metoda GetImages() trimite un request de tip GET către /api/image. Request-ul include token-ul JWT și răspunsul JSON primit de la server este deserializat într-un vector de obiecte ImageFileDTO. Pentru această operație este utilizată biblioteca Newtonsoft.Json, deoarece răspunsul serverului conține o colecție JSON de imagini.

```
2 references
public IEnumerator GetImages(
    System.Action<ImageFileDTO[]> callback)
{
    string url = ApiClient.BaseUrl + "/image";

    using var request = UnityWebRequest.Get(url);

    request.SetRequestHeader(
        "Authorization",
        "Bearer " + ApiClient.Token
    );

    yield return request.SendWebRequest();
}
```

Pentru vizualizarea unei imagini este utilizată metoda DecryptImage care construiește un request pentru POST /api/image/{id}/decrypt.


```

public IEnumerator DecryptImage(
    int id,
    string password,
    Action<Texture2D> callback)
{
    string url = ApiClient.BaseUrl + $"/image/{id}/decrypt";

    DecryptRequest body = new DecryptRequest
    {
        encryptionPassword = password
    };

    string json = JsonUtility.ToJson(body);
    byte[] bodyRaw = Encoding.UTF8.GetBytes(json);

    using UnityWebRequest request = new UnityWebRequest(url, "POST");

```

Duplicarea este realizată prin metoda DuplicateImage() care transmite POST /api/image/{id}/duplicate.

```

public IEnumerator DuplicateImage(
    int id,
    Action<bool> callback)
{
    string url = ApiClient.BaseUrl + $"/image/{id}/duplicate";

    using UnityWebRequest request = new UnityWebRequest(url, "POST");

    request.downloadHandler = new DownloadHandlerBuffer();

    request.SetRequestHeader(
        "Authorization",
        "Bearer " + ApiClient.Token
    );

```

Metoda ApplyFilter() transmite către server atât filtrul selectat cât și parola imaginii. Datele sunt trimise sub formă de FilterRequest către POST api/image/{id}/filter

```

1 reference
public IEnumerator ApplyFilter(
    int id,
    string filter,
    string password,
    Action<bool> callback)
{
    string url = ApiClient.BaseUrl + $"/image/{id}/filter";

    FilterRequest body = new FilterRequest
    {
        filter = filter,
        encryptionPassword = password
    };

    string json = JsonUtility.ToJson(body);
    byte[] bodyRaw = Encoding.UTF8.GetBytes(json);

```

Încărcarea imaginilor este realizată prin UploadImage(). Pentru upload este construit un request de tip multipart/form-data, format necesar pentru transmiterea simultană a fișierului și a câmpurilor text. Request-ul final este trimis către POST /api/image/upload.

```

1 reference
public IEnumerator UploadImage(
    string filePath,
    string subject,
    string password,
    Action<bool> callback)
{
    byte[] fileData = System.IO.File.ReadAllBytes(filePath);
    string fileName = System.IO.Path.GetFileName(filePath);

    List<IMultipartFormSection> formData =
        new List<IMultipartFormSection>();

    formData.Add(
        new MultipartFormFileSection(
            "file",
            fileData,
            fileName,
            GetContentType(filePath)
        )
    );
}

```

7.6 Gestionarea galeriei – Gallery UI

Clasa GalleryUI gestionează afișarea imaginilor în galerie și filtrarea acestora în funcție de subiect. La încărcarea scenei este solicitată lista imaginilor prin intermediul ImageService.

După primirea datelor, imaginile sunt păstrate local, este populată lista de subiecte și sunt generate cardurile.

```
Unity Script (1 asset reference) | 3 references
public class GalleryUI : MonoBehaviour
{
    [SerializeField]
    private ImageService imageService;
    [SerializeField]
    private Transform contentParent;
    [SerializeField]
    private GameObject imageCardPrefab;
    [SerializeField]
    private ActionsPanelUI actionsPanel;
    [SerializeField]
    private GalleryUI galleryUI;
    [SerializeField]
    private TMP_Dropdown subjectDropdown;

    private ImageFileDTO[] allImages;

    Unity Message | 0 references
    private void Start()
    {
        subjectDropdown.onValueChanged.AddListener(FilterBySubject);
        StartCoroutine(imageService.GetImages(OnImagesLoaded));
    }
}
```

Pentru fiecare imagine este instanțiat dinamic un imageCardPrefab. Astfel numărul elementelor din galerie este determinat automat de imaginile returnate de server. Subiectele existente sunt extrase din imaginile încărcate și introduse într-un TMP_Dropdown. Este adăugată și opțiunea All, care permite afișarea tuturor imaginilor.

```

foreach (var image in images)
{
    GameObject card = Instantiate(
        imageCardPrefab,
        contentParent
    );

    ImageCardUI cardUI =
        card.GetComponent<ImageCardUI>();

    cardUI.Setup(image, actionsPanel);
}

```

Subiectele existente sunt extrase din imaginile încărcate și introduse într-un TMP_Dropdown. Este adăugată și opțiunea All, care permite afișarea tuturor imaginilor. La selectarea unui subiect, imaginile sunt filtrate:

```

ImageFileDTO[] filteredImages =
    System.Array.FindAll(
        allImages,
        image => image.subject == selectedSubject
    );

DisplayImages(filteredImages);

```

Clasa ImageCardUI gestionează reprezentarea individuală a unei imagini în galerie. Fiecare card conține previzualizarea imaginii, subiectul acesteia și un buton pentru accesarea operațiilor disponibile.

Datele sunt asociate cardului prin metoda Setup()

```

public void Setup(
    ImageFileDTO image,
    ActionsPanelUI panel)
{
    imageData = image;
    actionsPanel = panel;

    subjectText.text = image.subject;

    actionsButton.onClick.AddListener(OpenActions);
}

```

1 reference

7.7 Panoul de acțiuni asupra imaginilor – ActionsPanelUI

Clasa ActionsPanelUI gestionează operațiile disponibile pentru imaginea selectată din galerie, făcând legătura dintre interfața utilizatorului și metodele definite în ImageService. La deschiderea panoului, metoda Open() păstrează referința către imaginea și cardul selectat, afișează subiectul imaginii și resetează câmpul destinat parolei de criptare. Butoanele pentru vizualizare, duplicare, aplicarea unui filtru, ștergere și închiderea panoului sunt asociate metodelor corespunzătoare în metoda Start().

```
Unity Message | 0 references
private void Start()
{
    closeButton.onClick.AddListener(Close);
    viewButton.onClick.AddListener(ViewImage);
    duplicateButton.onClick.AddListener(DuplicateImage);
    filterButton.onClick.AddListener(ApplyFilter);
    deleteButton.onClick.AddListener>DeleteImage);
}

1 reference
```

Pentru vizualizarea unei imagini, utilizatorul trebuie să introducă parola de criptare, aceasta fiind transmisă împreună cu identificatorul imaginii către metoda DecryptImage() din ImageService; dacă decriptarea este realizată cu succes, textura rezultată este transmisă către ImageCardUI și afișată în galerie.

```
1 reference
private void ViewImage()
{
    if (selectedImage == null)
        return;

    string password = passwordInput.text;

    if (string.IsNullOrEmpty(password))
    {
        Debug.LogWarning("Enter encryption password.");
        return;
    }

    StartCoroutine(
        imageService.DecryptImage(
            selectedImage.id,
            password,
            OnImageDecrypted
        )
    );
}
```

Duplicarea și ștergerea sunt realizate prin apelarea metodelor DuplicateImage() și DeleteImage(), iar după finalizarea cu succes a operației este apelată metoda RefreshGallery() pentru actualizarea conținutului galeriei.

```
private void DuplicateImage()
{
    if (selectedImage == null)
        return;

    StartCoroutine(
        imageService.DuplicateImage(
            selectedImage.id,
            OnImageDuplicated
        )
    );
}
```

Aplicarea unui filtru presupune selectarea acestuia dintr-un TMP_Dropdown și introducerea parolei de criptare, după care identificatorul imaginii, filtrul și parola sunt transmise către ImageService.ApplyFilter().

```
private void ApplyFilter()
{
    if (selectedImage == null)
        return;

    string password = passwordInput.text;

    if (string.IsNullOrEmpty(password))
    {
        Debug.LogWarning("Enter encryption password.");
        return;
    }

    string selectedFilter =
        filterDropdown.options[filterDropdown.value].text;

    StartCoroutine(
        imageService.ApplyFilter(
            selectedImage.id,
            selectedFilter,
            password,
            OnFilterApplied
        )
    );
}
```

1 reference

După generarea cu succes a imaginii filtrate, galeria este reîncărcată pentru afișarea noului rezultat. Astfel, ActionsPanelUI centralizează interacțiunile utilizatorului cu o imagine selectată și delegă operațiile efective către serviciul responsabil de comunicarea cu API-ul.

8. Securitatea Aplicației

Securitatea aplicației Photo Booth a fost implementată atât la nivelul autentificării utilizatorilor, cât și la nivelul accesului și stocării imaginilor. Principalele mecanisme utilizate sunt ASP.NET Core Identity pentru gestionarea conturilor, token-uri JWT pentru autorizarea cererilor, separarea imaginilor în funcție de utilizator și criptarea fișierelor înainte de stocarea acestora pe server.

Gestionarea utilizatorilor este realizată prin ASP.NET Core Identity, utilizând clasa ApplicationUser, derivată din IdentityUser. Înregistrarea unui utilizator este realizată prin UserManager.CreateAsync(), iar autentificarea presupune identificarea contului după adresa de e-mail și verificarea parolei prin UserManager.CheckPasswordAsync(). Astfel, aplicația nu implementează manual mecanismul de stocare și verificare a parolelor, acesta fiind gestionat de infrastructura Identity, care stochează parolele sub formă de hash și nu în clar.

După autentificarea cu succes, serverul generează un JSON Web Token (JWT) care conține identificatorul, numele și adresa de e-mail a utilizatorului. Token-ul este semnat folosind algoritmul HMAC-SHA256 și are o perioadă de valabilitate de o oră. Clientul Unity păstrează token-ul în ApiClient.Token și îl transmite ulterior către endpoint-urile protejate prin header-ul HTTP:

Authorization: Bearer <token>

Imaginile sunt criptate înainte de salvarea pe sistemul de fișiere. Parola introdusă de utilizator nu este utilizată direct drept cheie AES.

Din aceasta este derivată o cheie de 256 de biți prin PBKDF2, folosind Rfc2898DeriveBytes, SHA-256, un salt aleator de 16 bytes și 100.000 de iterații. Pentru criptarea efectivă este utilizat AES-GCM, fiind generate un nonce de 12 bytes și un tag de autentificare de 16 bytes.

AES-GCM permite atât protejarea confidențialității datelor, cât și verificarea integrității acestora. Fișierul rezultat în urma criptării este cel salvat în directorul Uploads; imaginea originală nu este salvată în clar în acest director.

Componenta server conține configurarea:

App.UseHttpsRedirection();

Clientul este configurat să comunice cu serverul prin:

<https://localhost:7043/api>

